BACHELOR'S THESIS (TFG) · UC3M

Four papers for RL in CARLA

What each paper does, what it shows, and how it applies to learning from every new flight

[Your name] · Supervisor: [name] · September 2026

All numbers were taken from the PDFs in `papers/`, with the table or section they come from. Text in green boxes is my own proposal for CARLA, not a result from the papers. Orange boxes are limits of the papers that matter for CARLA.

Contents

1. Why these four papers 1
2. TAILOR — the RL component 2
3. Simple Supervision Is Hard to Beat — how to use the labels 4
4. On Warm-Starting — how to retrain every round 5
5. LUF-net — making sunny and cloudy flights comparable 7
6. Putting the four together 8
7. Questions for the supervisor 9
References 9

1. Why these four papers

CARLA (github.com/jmartinpizarro/carla) detects cardilla (*Eryngium horridum*) in DJI drone video. The previous thesis trained YOLO-nano models once, on flights from a single date (test mAP50 0.81–0.85). This TFG turns that into a loop: after every uploaded flight, a person labels a few tiles and the detector is updated. Flights are manual, processing is offline, and there is one desktop GPU.

A loop like that has to answer four questions. Each paper answers one:

Question in the loop	Paper	Answer it gives
Which tiles should a person label after each flight?	**TAILOR** (NeurIPS 2023)	A bandit (RL) learns which labeling strategy to use, with a reward computed from the new labels.
How should the few labels be used together with the unlabeled footage?	**Simple Supervision** (arXiv 2026)	Add them as plain supervised examples next to pseudo-label training (RTSM); clever tricks rarely add more.
How should the detector be retrained after every flight?	**On Warm-Starting** (NeurIPS 2020)	Not by simply continuing from the last weights; shrink and perturb them first.
How can sunny and cloudy flights be made comparable?	**LUF-net** (Plant Phenomics 2026)	Correct colour with a network conditioned on the camera's exposure settings (and light sensor).



One round of the loop, run after every uploaded flight. Only the second box is reinforcement learning.

Where the RL is. Only TAILOR is reinforcement learning: a multi-armed bandit, the simplest RL setting, with exploration (trying strategies) and exploitation (using the ones that pay off). The other three papers do not use RL, but without them the loop would either waste its labels (Simple Supervision), degrade with every retrain (Warm-Starting), or confuse weather changes with detector errors (LUF-net).

2. TAILOR — the RL component

Zhang, Shao, Verma & Nowak. *Algorithm Selection for Deep Active Learning with Imbalanced Datasets.* NeurIPS 2023. Code: github.com/jifanz/TAILOR.

The problem

Active learning picks which unlabeled examples a person should label next. Dozens of strategies exist (least confident, most diverse, BADGE, GALAXY…), and which one works best "can vary dramatically across datasets and applications", so it cannot be chosen in advance. Earlier methods that pick strategies online were built for linear models. Their rewards were based on training accuracy, which is almost always 100% for deep networks, so it gives no signal (Section 2).

The method

TAILOR (*Thompson ActIve Learning algORithm selection*) turns strategy selection into a **multi-armed bandit**:

- Each of the M candidate strategies is an **arm**. In one round, B examples are labeled; for **each slot** in the batch the bandit picks an arm, and that strategy picks one example. One batch can therefore mix strategies.
- The **reward** is computed only from the labels of the picked examples, as soon as they are annotated, with no retraining. For class diversity (Section 4.1):

$$r^t_{\text{div}}(x, y) = \frac{1}{K}\sum_{i=1}^{K} \frac{y_i}{\max(1, \text{COUNT}_t(i))} = \langle v^t_{\text{div}}, y \rangle$$

  where $\text{COUNT}_t(i)$ is how many labeled examples of class i exist so far. A label from a still-rare class is worth more. The paper also allows **domain-specific** weights v^t_{dom} chosen by experts and changed over time.
- **Thompson sampling** (Algorithm 2): each arm keeps a Beta posterior per class (Dirichlet for multi-class). For each slot, sample $\hat{\theta}_i \sim \text{Beta}(a_i, b_i)$ for every arm and pick $\arg\max_i \langle v^t, \hat{\theta}_i \rangle$. After labeling, update $a_i \leftarrow \gamma a_i + \sum y$ and $b_i \leftarrow \gamma b_i + \sum(1-y)$ over the examples arm i picked. The discount $\gamma = 0.9$ handles rewards that change over time (Section 6.1).
- Then the network is retrained on all labels and the next round starts.

The regret bound is $\tilde{O}\left(B\sqrt{MT}\right)$, better than treating it as a generic linear contextual bandit (Theorem 5.2).

Experimental setup

Ten imbalanced image **classification** datasets: four multi-label (CelebA, COCO, VOC, Stanford Car) and six multi-class (ImageNet, Kuzushiji-49, Caltech256, and imbalanced versions of CIFAR-10, CIFAR-100, SVHN). Batch sizes of 500 to 10,000 labels per round, only 10 or 20 rounds, between 7 and hundreds of candidate strategies, four trials each (Section 6.1). Baselines: every single strategy, a **Random Meta** selector, and **ALBL**, the earlier accuracy-rewarded bandit.

Results

- Against the other meta-selectors, TAILOR is better on **all** datasets, in both accuracy and class diversity.
- **ALBL performed like Random Meta on all datasets**: accuracy-based rewards did not help.
- Against the best single strategy: on par in accuracy on **9 of 10** datasets (the exception is Caltech256), on par in class diversity on all 10, and better in class diversity on 4 of 10. On CelebA it beats the best single strategy by a significant margin, by combining several strategies (Section 6.2).
- With the "search" reward (count of positive labels), it finds the most positives on 3 of 4 multi-label datasets and is second on the fourth.
- Runtime: retraining dominates; TAILOR itself was more than 20% faster than the slowest candidate strategy (Section 5.1).

Caveat: Only classification was tested, never object detection. Rounds used 500–10,000 labels each; CARLA would label tens of tiles per flight, so the posteriors will learn from far fewer observations. The authors themselves list non-stationary label distributions as future work, and flights in different weather are exactly that.

How it applies to CARLA

Each uploaded flight is one round. Candidate arms for CARLA:

1. Every 3 s (what the previous thesis did)
2. Random
3. Least confident tiles
4. Most different from labeled tiles
5. Plants that flicker between adjacent video frames
6. Disagreement between YOLO and T-Rex2
7. Tiles from under-covered altitude × light conditions

My proposal (not from the papers): CARLA has a single class, so class diversity makes no sense directly. But TAILOR's **multi-label** formulation fits once each labeled tile is described by a binary vector y that is known right after annotation: *contains cardilla*, *detector missed a plant*, *detector produced a false positive*, plus a one-hot *altitude × light cell* from the drone telemetry. The reward $\langle v^t, y \rangle$ with inverse-count weights $v_i^t = \frac{1}{K \max(1, \mathrm{COUNT}_t(i))}$ is exactly TAILOR's form, so its Beta posteriors and Thompson sampling can be used unchanged. Rare detector errors in rare conditions become the most valuable labels.

What to measure: the number of labeled tiles needed to reach a target mAP50 on held-out flights, for the bandit against each single arm and against random. Also compare this label-based reward against a Δ mAP reward. TAILOR's ALBL result predicts the label-based reward will win.

3. Simple Supervision Is Hard to Beat — how to use the labels

Zhang, Xu & Agrawal (Amazon Robotics). *Simple Supervision Is Hard to Beat: A Bitter Lesson from Sparse Target Labels in Domain-Adaptive Object Detection.* arXiv 2606.30795, June 2026 (preprint).

The problem

A detector trained on one domain is adapted to a new one (fog, other cities, other cameras) **without the source data** ("source-free"). The usual way is **Mean Teacher self-training**: a teacher model predicts pseudo-labels on unlabeled target images, a student learns from them, and the teacher is an exponential moving average of the student. Pseudo-labels contain false positives and miss hard objects. The paper asks: if a person labels a small **random** subset of target images, what is the best way to use those labels?

The random choice is deliberate: the authors say it "is not meant to be the most label-efficient annotation strategy", but it isolates the question of **using** labels from the question of **choosing** them (Section 1). TAILOR answers the second question, so the two papers fit together.

The method

RTSM (*Random-Target Supervised Mixing*) keeps the self-training branch $\mathcal{L}_u$ unchanged and adds the ordinary detection loss on the labeled batch (Eq. 4):

$$\mathcal{L}_{\text{RTSM}} = \mathcal{L}_u + \frac{1}{|\mathcal{B}_l|} \sum_{(x,y)\in\mathcal{B}_l} \mathcal{L}_{\text{det}}(f_\theta(x), y)$$

Against this anchor they test **ten plugins** that use the same labels to steer the unlabeled branch (Section 3):

- **Pseudo-label selection** (precision): prior-mapped thresholds, precision-calibrated thresholds, reliability score rescaling.
- **Object completion** (recall): hard query recovery, multi-view hard recovery, foreground query revival, multi-view foreground revival.
- **Optimization control**: loss balancing, target-anchored PCGrad, label-guided teacher update.

Experimental setup

Cityscapes → Foggy Cityscapes and Cityscapes → BDD100K (daytime), with two transformer detectors (DINO and DETA, ResNet-50 backbone), four source-free methods (PETS, LPU, LPLD, DDT), and label budgets of 1%, 5% (default) and 10% of target images. Metric: AP50.

Results

RTSM against pure source-free adaptation, Foggy Cityscapes, DINO, 5% labels (Table 1):

AP50	PETS	LPU	LPLD	DDT
Pure SFDA	46.5	48.7	47.0	53.7
RTSM	**52.9**	**53.4**	**55.3**	**58.7**
Gain	+6.4	+4.7	+8.3	+5.0

References: source-only 32.6, fully labeled target (oracle) 62.5.

- Across all settings, RTSM improves pure source-free adaptation by **+1.7 to +18.3 AP50** (abstract). On BDD100K the gains are +4.4 to +18.3 (Section 4.3).
- **No plugin consistently beats RTSM.** The best plugin changes with the method: precision thresholds for PETS, score rescaling for LPU, multi-view foreground revival for LPLD, PCGrad for DDT.
- Plugins can hurt badly: **multi-view hard recovery lost 13.6 AP50** on PETS, Cityscapes → BDD100K (Table 3); with DETA, hard recovery lost up to 10.9 (Table 4).

- Gains are fragmented across classes: a plugin that helps *person* and *bus* hurts *truck* and *bicycle* (Table 2).
- More labels, less room for tricks: for DDT, RTSM goes 55.4 → 58.7 → 59.9 AP50 at 1% → 5% → 10%, and the average plugin effect goes from +0.10 to −0.86 and −0.82 (Section 4.3).
- The most stable plugin was multi-view **foreground revival** (a soft loss, not new hard pseudo-labels): +0.2 to +0.8 on all four methods with DETA.

Caveat: Preprint. Transformer detectors on street scenes; the authors state that other detector families were not studied. Labeled images were chosen at random, not actively.

How it applies to CARLA

- **Training recipe for every round:** pseudo-labels on the unlabeled footage of the flight, plus the newly labeled tiles as ordinary supervised examples.
- **The baseline the bandit must beat:** random tiles with this same recipe. If TAILOR's tiles do not beat random tiles under RTSM, the RL component adds nothing.
- **A warning for multi-frame filtering.** Keeping only detections that persist across video frames is close to the paper's "multi-view" plugins. The hard version was among the worst; the soft version was the most stable.

My proposal (not from the papers): Use multi-frame agreement as a soft signal (a revival-style loss or a lower weight) rather than to create hard pseudo-labels, and keep it only if it beats plain RTSM on held-out flights.

4. On Warm-Starting — how to retrain every round

Ash & Adams. *On Warm-Starting Neural Network Training.* NeurIPS 2020 (arXiv 1910.08475).

The problem

When data arrive in pieces, it seems natural to continue training from the previous model ("warm start") instead of starting from a random initialization. For convex models that works. For deep networks the paper shows that warm-starting **hurts test accuracy even though training accuracy is the same**. The authors note that deep active learning code usually retrains from scratch after every labeling step for this reason (Section 1).

The experiments

Train to convergence on 50% of the data, then on 100% starting from those weights, and compare with a fresh random initialization on 100% (Table 1). Learning rate 0.001, batch 128, **no data augmentation and no learning-rate schedule** (Section 3).

Test accuracy (%)	ResNet-18, SGD	ResNet-18, Adam	Logistic regression, SGD
CIFAR-10: random init	56.2	78.0	40.5
CIFAR-10: warm start	51.7	74.4	39.6
CIFAR-100: random init	18.2	41.4	16.9
CIFAR-100: warm start	15.5	35.0	16.3
SVHN: random init	89.4	93.6	28.0
SVHN: warm start	87.5	93.5	28.0

- The damage is consistent for ResNets and MLPs, larger on harder data (CIFAR) and absent for convex logistic regression.
- In an **online** experiment (1,000 new CIFAR-10 images per round), the gap **grows as more data arrive** (Fig. 2).
- Larger learning rates, smaller batches or regularization (L2, confidence penalty, adversarial training) can close the gap only by making training as slow as starting from scratch (Sections 3.1, 3.3).
- A few epochs of first-round training are already enough to cause the damage (Section 3.2).
- Explanation: in a warm-started model, gradients from new data are much larger than from data already seen, so the two halves are not learned the same way (Fig. 5).

The fix: Shrink & Perturb

Before each round, shrink the weights and add noise (Section 4):

$$\theta_i^t \leftarrow \lambda \theta_i^{t-1} + p^t, \quad p^t \sim \mathcal{N}(0, \sigma^2), \quad 0 < \lambda < 1$$

In practice the noise is taken from a scaled, freshly initialized network. Shrinking keeps what the network predicts (it makes the model less confident, not different) and balances the gradients of old and new data.

- **Online learning:** $\lambda = 0.6$, noise scale 0.01 matches random initialization in accuracy **while training much faster**; smaller λ can even beat it (Fig. 7).
- **Pre-training:** when starting from weights trained on another dataset, warm start is sometimes better and sometimes worse than training from scratch, and the crossover cannot be predicted. Shrink & Perturb ($\lambda = 0.3$, noise 10^{-4}) was at least as good as the better of the two (Section 4.2, Fig. 9).

Caveat: Image classification only, without augmentation or learning-rate schedules. YOLO training uses heavy augmentation, cosine or linear schedules and warm-up, which may already reduce the gap. The effect on CARLA's detector must be measured, not assumed.

How it applies to CARLA

- Every uploaded flight means a retrain. Training from scratch every round (the previous thesis used 200 epochs × 5 seeds) is too slow on one GPU; simply continuing from the last weights may slowly degrade the detector.
- CARLA's YOLO starts from COCO-pretrained weights, which is the paper's **pre-training** case, and then keeps receiving flights, which is its **online** case. Shrink & Perturb covers both.
- It is a few lines of code applied to the checkpoint before each round.

My proposal (not from the papers): Compare four update rules per round: warm start, Shrink & Perturb with $\lambda \in \{0.3, 0.6\}$, and restart from COCO weights, recording held-out mAP50 and GPU time. If the answer changes over rounds, the update rule can become a second small bandit arm.

5. LUF-net — making sunny and cloudy flights comparable

Wang, Liu, Zhang, Jin & Yuan. *LUF-net: A physically informed color calibration method for UAV RGB images based on exposure and irradiance information.* Plant Phenomics 8 (2026) 100262 (doi). Code and weights: github.com/wangchufeng3652/color-correction.

The problem

Drone RGB images do not record the true colour of plants. The recorded colour changes with the sun, clouds, and the camera's automatic ISO, shutter speed and aperture, so the same field looks different from flight to flight. Classic colour constancy (Gray-world) assumes uniform light; image-only deep networks ignore the metadata that actually causes the change (Section 1).

The method

- **Data:** 230 images from a DJI Mavic 3 Multispectral (5472 × 3648) over maize, soybean and rice, at 6, 12, 25 and 40 m, under overcast, cloudy, sunny and early-morning light; cut into 58,292 patches of 256 × 256 px. Train, validation and test (60/20/20) were split **by original image**, not by patch, to avoid leakage (Section 2.2).
- **Ground truth:** a 24-patch colour chart measured with a colorimeter; a linear regression maps observed to true colours in CIE Lab (R^2 0.98–0.99, Section 3.1).
- **Network:** a light U-net (encoder with 32, 64, 128 channels and a 256-channel bottleneck) working in Lab space. A 7-value metadata vector (aperture, exposure time, ISO, and irradiance in the G, R, red-edge and NIR bands from the drone's light sensor) goes through a small MLP and modulates the features after each encoder block with FiLM (Eq. 12):

$$F' = F \cdot (1 + \gamma(V)) + \beta(V)$$

Results

Method (Table 1 and Fig. 6)	Colour error	MAPE
Gray-world	26.1	14.6%
U-net, image only	16.7	9.31%
AlexNet-MLP, with metadata	14.1	7.89%
SIDBlock-MLP, with metadata	13.6	7.6%
LUF-net, exposure only	8.86	**4.95%**
LUF-net, irradiance only	7.34	4.10%
LUF-net, both but without FiLM	8.09	4.52%
LUF-net, full	**6.96**	**3.89%**

By condition (Table 2), MAPE was 3.53% overcast, 3.63% early morning, 4.19% cloudy and **5.30% sunny**, the hardest because of specular reflections and shadows. By altitude it ranged from 2.72% (25 m) to 4.98% (40 m). The link between error and acquisition settings dropped from Pearson r = 0.20–0.28 (U-net) to

about 0.07–0.08 (LUF-net, Section 3.5). The model has 0.95M parameters and takes 1.27 s per 100 patches on an RTX 2080 Ti.

Caveat: It measures colour accuracy for phenotyping, never detection. Training needs flights with a colour chart. It was tested on still images of crops from one drone with a light sensor, not on compressed video of pasture.

How it applies to CARLA

- **For the generalization goal:** a preprocessing step on the server that makes sunny and overcast flights look alike before detection. The exposure-only variant needs no extra hardware: DJI video subtitles (`.SRT`) normally log ISO, shutter and aperture per frame. This has to be checked on CARLA's own files.
- **For the RL loop:** the same telemetry gives every tile an altitude × light cell. That cell is part of the TAILOR reward vector and drives the "under-covered condition" arm (Section 2).
- **Cost:** at 1.27 s per 100 patches, one 1080p frame (about 32 patches) takes about 0.4 s. Too slow for live video, but fine for offline processing of sampled frames.

My proposal (not from the papers): Test whether colour correction closes the detector's sunny-vs-overcast gap: train on sunny flights, test on overcast ones, and compare (a) no correction, (b) photometric augmentation during training, (c) LUF-net exposure-only preprocessing. Option (b) is the cheap baseline that (c) must beat.

6. Putting the four together

Paper	What CARLA takes from it	Experiment in the thesis	Main risk
TAILOR	Bandit over labeling strategies with a label-based reward, redefined for one class	Labels needed to reach a target mAP: bandit vs. each arm vs. random	Few labels per round; untested on detection
Simple Supervision	RTSM training recipe; random tiles + RTSM as the baseline	With vs. without pseudo-labels; hard vs. soft multi-frame filtering	Transformers on street scenes, not YOLO on drones
Warm-Starting	Shrink & Perturb before each retrain	Warm start vs. S&P vs. restart: accuracy and GPU time per round	Augmentation and LR schedules may change the effect
LUF-net	Exposure-conditioned colour correction; condition labels from telemetry	Train sunny, test overcast: none vs. augmentation vs. LUF-net	Colour only; needs a colour chart

One round, concretely: a flight is uploaded with its telemetry → frames are colour-corrected and assigned an altitude × light cell → the detector runs on overlapping tiles → TAILOR picks k tiles and a person labels them → rewards are computed from those labels and the bandit's posteriors are updated → the checkpoint is shrunk and perturbed → the detector is fine-tuned on all labels plus pseudo-labels (RTSM) → evaluated on held-out flights, reporting the worst condition.

A shared lesson. Three of the four papers make the same point: simple, well-chosen signals beat elaborate ones. A label-based reward beats an accuracy-based one (TAILOR vs. ALBL). Plain supervision beats label-guided pseudo-label tricks (RTSM). A simple rescaling beats hyperparameter tuning (Shrink & Perturb). The thesis should therefore always report the simple baselines next to the RL component.

7. Questions for the supervisor

1. Is a bandit (the simplest form of RL) enough RL for the thesis, or is a learned policy (e.g. policy gradient over tiles, as in MGRAL, ICASSP 2026) expected?
2. What GPU and VRAM will be used for training? This decides how many simulated rounds are affordable.
3. How many flights and labeled tiles exist, and can I fully label one or two flights to simulate the loop (hide labels, reveal them when a strategy asks)?
4. Which drone is used, and are the `.SRT` files uploaded with the videos? Could a colour chart be placed in a few flights?

References

- Zhang, J., Shao, S., Verma, S. & Nowak, R. Algorithm Selection for Deep Active Learning with Imbalanced Datasets. *NeurIPS* 2023. proceedings.neurips.cc
- Zhang, L., Xu, R. & Agrawal, M. Simple Supervision Is Hard to Beat: A Bitter Lesson from Sparse Target Labels in Domain-Adaptive Object Detection. arXiv 2606.30795, 2026. arxiv.org/abs/2606.30795
- Ash, J. T. & Adams, R. P. On Warm-Starting Neural Network Training. *NeurIPS* 2020. arxiv.org/abs/1910.08475
- Wang, C., Liu, B., Zhang, J., Jin, S. & Yuan, F. LUF-net: A physically informed color calibration method for UAV RGB images based on exposure and irradiance information. *Plant Phenomics* 8, 100262, 2026. doi.org/10.1016/j.plaphe.2026.100262
- Liang et al. Performance-guided Reinforced Active Learning for Object Detection (MGRAL). *ICASSP* 2026. arxiv.org/abs/2601.15688 (mentioned in section 7 only)